

AI/ML Internship – Day 70 Notes & Tasks

Module 9: Building a Complete Retrieval-Augmented Generation (RAG) Pipeline

Part 1: Recap of Day 69

Yesterday we learned:

-  Vector Databases
-  ChromaDB
-  FAISS
-  Similarity Search

Today we will combine everything and build a complete RAG workflow.

Today's topics:

- Complete RAG Pipeline
 - Document Processing
 - Retrieval Process
 - Response Generation
 - Real-World Architecture
-

Part 2: What is a Complete RAG Pipeline?

A **RAG Pipeline** is a sequence of steps that allows an AI chatbot to answer questions using external documents.

Instead of relying only on its built-in knowledge, the chatbot retrieves relevant information from documents before generating a response.

Simple Definition

RAG = Retrieve + Generate

1. Retrieve relevant information.
 2. Generate the final answer.
-

Part 3: Complete Workflow

Upload PDF



Extract Text from PDF



Split into Chunks



Generate Embeddings



Store in Vector Database



User Question



Convert Question to Embedding



Similarity Search



Retrieve Relevant Chunks



Send Context + Question to LLM

|



Generate Final Response

|



Display to User

Part 4: Step 1 – Uploading Documents

The first step is to upload documents into the system.

Examples:

- PDF
- Word Document
- TXT File
- Excel Sheet
- Company Policy
- User Manual

Example:

Employee_Handbook.pdf

Admission_Guide.pdf

Part 5: Step 2 – Extracting Text

The chatbot extracts readable text.

Example:

PDF



Internship Duration: 3 Months

Eligibility: Plus Two

Fee: ₹5000

Libraries used:

- PyPDF2
- pdfplumber
- PyMuPDF

Part 6: Step 3 – Chunking

Large documents are divided into smaller pieces.

Example:

Chunk 1

Internship Duration

Chunk 2

Eligibility

Chunk 3

Placement

Chunk 4

Fee Structure

Why?

Large Language Models cannot efficiently process hundreds of pages in one request.

Part 7: Step 4 – Creating Embeddings

Each chunk is converted into a numerical vector.

Example:

Chunk

Course Fee is ₹5000

↓

Embedding

[0.81, -0.22, 0.17, 0.64, ...]

Library:

```
from sentence_transformers import SentenceTransformer
```

Part 8: Step 5 – Store in ChromaDB

Example:

```
collection.add(  
    documents=[  
        "Fee is ₹5000",  
        "Duration is 3 Months",  
        "Placement Assistance"  
    ],  
    ids=["1","2","3"]  
)
```

Now all embeddings are stored.

Part 9: Step 6 – User Asks a Question

Example:

How much is the course fee?

The chatbot converts the question into an embedding.

Part 10: Step 7 – Similarity Search

The vector database compares:

Question Embedding

↓

Stored Embeddings

↓

Finds the closest document.

Example:

Retrieved Chunk

Course Fee is ₹5000

Part 11: Step 8 – Sending Context to the LLM

The chatbot sends:

Question

-

Retrieved Document

Example:

Context:

Course Fee is ₹5000.

Question:

How much is the fee?

The LLM now has the correct information.

Part 12: Step 9 – Response Generation

The AI model generates a natural language response.

Example:

The AI Internship fee is ₹5000.

Notice:

The answer comes from the retrieved document, not from guessing.

Part 13: Complete Python Workflow (Concept)

Step 1

Read PDF

Step 2

Extract Text

Step 3

Split into Chunks

Step 4

Generate Embeddings

Step 5

Store in ChromaDB

Step 6

Accept User Question

Step 7

Retrieve Similar Chunks

Step 8

Send Context to OpenAI

Step 9

Display Response

Suppose your company has:

- HR Policy
- Leave Policy
- Internship Rules
- Salary Guidelines

Employee asks:

How many leave days do interns get?

The chatbot:

- Searches HR Policy
 - Retrieves the relevant section
 - Sends it to the LLM
 - Generates the answer
-

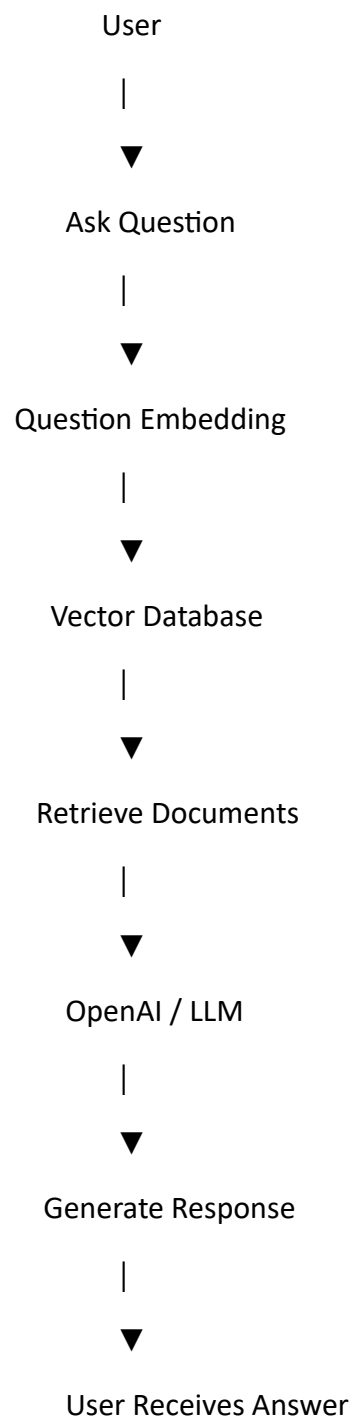
Part 15: Advantages of RAG

- ✓ Uses company documents
 - ✓ Latest information
 - ✓ Easy to update
 - ✓ More accurate answers
 - ✓ Reduces hallucinations
 - ✓ No need to retrain the model
-

Part 16: Limitations

- ✗ Poor documents produce poor answers
 - ✗ Incorrect chunking reduces accuracy
 - ✗ Requires embedding models
 - ✗ Requires vector databases
-

📌 Part 17: Complete RAG Architecture



📌 Part 18: Theory Questions

What is a RAG Pipeline?

A RAG pipeline is a workflow that retrieves relevant information from external knowledge sources and uses a Large Language Model to generate accurate responses.

Why is chunking required?

Chunking divides large documents into smaller pieces, making retrieval faster and more accurate.

What is the role of a vector database?

It stores embeddings and performs similarity searches to retrieve relevant document chunks.

Why are embeddings generated?

Embeddings convert text into vectors so that semantic similarity can be measured.

Why is RAG better than using only an LLM?

RAG allows the LLM to answer using up-to-date, organization-specific information instead of relying only on pre-trained knowledge.

Part 19: Interview Questions

Explain the complete RAG pipeline.

A RAG pipeline extracts text from documents, splits it into chunks, creates embeddings, stores them in a vector database, retrieves relevant chunks based on the user's query, and uses an LLM to generate the final response.

What happens after similarity search?

The retrieved document chunks are provided to the LLM as context for generating an accurate answer.

Why does RAG reduce hallucinations?

Because the LLM answers using retrieved factual information instead of relying only on its internal knowledge.

Which libraries are commonly used in a RAG project?

- PyPDF2
 - Sentence Transformers
 - ChromaDB
 - FAISS
 - OpenAI SDK
-

What are the major components of a RAG system?

- Document Loader
 - Text Splitter
 - Embedding Model
 - Vector Database
 - Retriever
 - Large Language Model (LLM)
-

Tasks for Day 70

Theory

1. Explain the complete RAG pipeline.
 2. Why are embeddings important?
 3. Explain similarity search.
 4. What is the role of an LLM in RAG?
 5. Why is RAG widely used in companies?
-

Practical Task 1

Draw the complete RAG architecture.

Practical Task 2

Prepare a flowchart showing:

PDF → Text → Chunks → Embeddings → ChromaDB → Retrieval → LLM → Answer

Practical Task 3

List the Python libraries required to build a RAG chatbot.

Practical Task 4

Create a table explaining the role of each component:

Component	Purpose
PDF Loader	Reads PDF files
Chunker	Splits text
Embedding Model	Converts text into vectors
ChromaDB	Stores vectors
Retriever	Finds similar chunks
LLM	Generates the final answer

Practical Task 5

Research two frameworks used for building RAG applications:

- LangChain
- LlamaIndex

Write a short note (5–6 lines) on each.